09/10/24

Genetic optimization of the Population

algorithm

Step1: Select a population of any size as sample

Step2: use chromosomes of the population

Step3: evaluate the fitness of chromosomes using fitness function

Step4: the value of of function are stored, select the best values (optimal)

Step5: If step4 results match to the termination condition stop the process.

Step6: select the values which are optimation from previous cross over them → combine two chromosomes for getting unique characteristic

Step7: mutate the chromosome results in divergence / diversity.

Step8: select the optimal population ~~and~~ from current and parent generation.

Step9: use steps 6 to 8 repeat.

Step10 produce the offspring from optimal solution

9/10/24

Tigers population v/s genetic optimization fitness.

Tiger breed with unknown mates in there locality but because of no natural pathways they face inbreeding depression, leads to less fertility. using this algorithm we can minimize the inbreeding depression & minimize probability of tiger movement to high density / over populated areas.

area, locality Ecosystem, inbreeding depression Biological Main

Step1: select random tiger ~~popopu~~ population

Step2: read the chromosome values for e

Step3: apply fitness values by fitness function.
calculate the scores for each tiger.

Step4: see if there is tiger with same score
yes - mates them as optimal solution for offsprings

Step5: if no them choose more suitable one which
contribute to fitness values

Step6: use the optimal tigers for crossover and produce
new generation

The generation produce it fittes to environment then
previous generation.

```
import random
genelength = 8
mutation_rate = 0.05
optimum = [1, 1, 0, 0, 1, 0, 0]
class Tiger:
    def __init__(self, dna):
        self.dna = dna
    def fitness(self):
        matches = sum(a == b for a, b in zip(self.dna, optimum))
        return (matches / genelength) * 100

def compatibility(male, female):
    differences = (sum(a != b for a, b in zip(male.dna, female.dna))
    return (differences / genelength) * 100

def crossover(male, female):
    point = random.randint(1, genelength - 1)
    child_dna = male.dna[:point] + female.dna[point:]
    return Tiger(child_dna)

def mutate(child):
    for i in range(gene_length):
```

```
    if random.random() < mutation_rate:
        child.dna[i] = 1 - child.dna[i]

def stability(child):
    return child.fitness()

male = Tiger([1,1,0,0,1,0,1,0])
female = Tiger([1,0,0,1,1,1,0,0])
print("Male DNA:", male.dna)
print("female DNA:", female.dna)
score = compatibility(male, female)
print("Genetic Diversity score:", score, "%")
child = crossover(male, female)
print("offspring after crossover:", child.dna)
mutate(child)
print("offspring after mutation:", child.dna)
print("offspring stability score:", stability(child), "%")
print("offspring fitness:", child.fitness(), "%")
```

Output

Male DNA = [1,1,0,0,1,0,1,0]
Female DNA = [1,0,0,1,1,1,0,0]
Genetic diversity score : 50.0%
Off spring after crossover = [1,1,0,0,1,1,0,0]
Off spring after mutation = [1,1,0,0,1,1,0,0]
Off spring stability score : 100.0%
Off spring fitness : 87.5%

Untitled1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[1] ✓ 0s

```
import random

GENE_LENGTH = 8
MUTATION_RATE = 0.05

# Ideal genes required for survival
OPTIMUM = [1, 1, 0, 0, 1, 1, 0, 0]


class Tiger:
    def __init__(self, dna):
        self.dna = dna

    def fitness(self):
        matches = sum(a == b for a, b in zip(self.dna, OPTIMUM))
        return (matches / GENE_LENGTH) * 100


def compatibility(male, female):
    differences = sum(a != b for a, b in zip(male.dna, female.dna))
    return (differences / GENE_LENGTH) * 100


def crossover(male, female):
    point = random.randint(1, GENE_LENGTH - 1)

    child_dna = male.dna[:point] + female.dna[point:]
    return Tiger(child_dna)


def mutate(child):
    for i in range(GENE_LENGTH):
        if random.random() < MUTATION_RATE:
            child.dna[i] = 1 - child.dna[i]
```

Untitled1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[1] ✓ 0s

```
def stability(child):
    return child.fitness()


# Main program
male = Tiger([1, 1, 0, 0, 1, 0, 1, 0])
female = Tiger([1, 0, 0, 1, 1, 1, 0, 0])

print("Male DNA:", male.dna)
print("Female DNA:", female.dna)

score = compatibility(male, female)
print("Genetic Diversity Score:", score, "%")

child = crossover(male, female)
print("Offspring after crossover:", child.dna)

mutate(child)
print("Offspring after mutation:", child.dna)

print("Offspring Stability Score:", stability(child), "%")
print("Offspring Fitness:", child.fitness(), "%")
```

```
Male DNA: [1, 1, 0, 0, 1, 0, 1, 0]
Female DNA: [1, 0, 0, 1, 1, 1, 0, 0]
Genetic Diversity Score: 50.0 %
Offspring after crossover: [1, 1, 0, 0, 1, 0, 0, 0]
Offspring after mutation: [1, 1, 0, 0, 1, 0, 0, 0]
Offspring Stability Score: 87.5 %
Offspring Fitness: 87.5 %
```